

CV9: Trvalé uloženie údajov pomocou Room - Teória

1 Úvod do cvičenia

V predchádzajúcom cvičení ste pracovali s `ViewModel` triedou ako s miestom, kde sa nachádza stav obrazovky a logika spracovania používateľských udalostí. Takýto prístup je dôležitý, pretože oddeľuje používateľské rozhranie od logiky aplikácie. Stále však platí, že dáta uložené iba vo `ViewModel` triede sa po ukončení aplikácie stratia. Ak teda chcete vytvoriť aplikáciu, ktorá si má zapamätať používateľom zadané údaje aj po zatvorení, potrebujete trvalé uloženie údajov.

V tomto cvičení preto prejdete od dočasného stavu obrazovky k lokálnej perzistencii. Budete vytvárať aplikáciu *Movie Watchlist*, v ktorej si používateľ ukladá vlastný zoznam filmov. Každý film bude možné pridať, zobrazíť, označiť ako videný, doplniť k nemu hodnotenie a komentár, prípadne ho odstrániť. Údaje sa budú ukladať do lokálnej databázy pomocou knižnice Room.

2 Prečo už nestačí iba lokálny stav v Compose

Pri jednoduchých obrazovkách často postačuje, ak si uchováate údaje v `remember`, `mutableStateOf` alebo vo `ViewModel` triede. Takéto riešenie je vhodné vtedy, keď má stav význam len počas aktuálneho používania obrazovky. Ak však aplikáciu zavriete a znovu otvoríte, údaje vytvorené iba týmto spôsobom sa nenačítajú späť.

Pri watchliste filmov by to bolo nepraktické. Používateľ očakáva, že zoznam filmov zostane zachovaný. Očakáva tiež, že ak už pri niektorom filme zadal komentár a hodnotenie, tieto údaje budú k dispozícii aj neskôr. Práve preto sa v tomto cvičení používa databáza. Používateľské rozhranie zostane v Compose, logika obrazovky ostane vo `ViewModel` triede, ale samotné údaje budú uložené v Room databáze.

3 Čo je Room

Room je knižnica z Android Jetpacku určená na prácu s lokálnou databázou SQLite cez vyššiu, pohodlnejšiu vrstvu. Namiesto priameho písania nízkoúrovňového databázového kódu budete pracovať s anotáciami a dátovými triedami. V praxi to znamená, že štruktúru tabuľky popíšete triedou, operácie nad databázou popíšete rozhraním a Room za vás pripraví potrebnú implementáciu.

V tomto cvičení budete používať Room preto, že ide o prirodzený nástroj na ukladanie štruktúrovaných údajov. Film v watchliste má viac vlastností. Nestačí teda uložiť len jedinú hodnotu. Každý záznam bude obsahovať názov filmu, informáciu o tom, či bol videný, hodnotenie, komentár a časové údaje. To je presne situácia, v ktorej databáza dáva zmysel.

4 Základné stavebné prvky Room

4.1 Entity

Entity predstavuje dátový model uložený v databáze. Môžete si ju predstaviť ako definíciu jednej tabuľky. Každý objekt tejto triedy zodpovedá jednému riadku tabuľky.

V aplikácii *Movie Watchlist* bude entita reprezentovať jeden film. Obsahovať bude identifikátor, názov, stav videnia, hodnotenie, komentár, čas pridania a čas označenia za videný.

```
@Entity(tableName = "movies")
data class MovieEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val title: String,
    val isWatched: Boolean = false,
    val rating: Int? = null,
    val comment: String = "",
    val createdAt: Long,
    val watchedAt: Long? = null
)
```

Všimnite si, že id sa generuje automaticky. Vďaka tomu používateľ nemusí zadávať žiadny identifikátor ručne. Pole `createdAt` si zapamätá čas pridania filmu a pole `watchedAt` bude vyplnené len pri filmoch, ktoré boli označené ako videné.

4.2 DAO

DAO je skratka pre *Data Access Object*. Ide o rozhranie, v ktorom definujete operácie nad databázou. Sem patrí načítanie údajov, vloženie nového záznamu, úprava existujúceho záznamu a odstránenie záznamu.

```
@Dao
interface MovieDao {

    @Query("SELECT * FROM movies ORDER BY createdAt DESC")
    suspend fun getAllMovies(): List<MovieEntity>

    @Insert
    suspend fun insertMovie(movie: MovieEntity)

    @Update
    suspend fun updateMovie(movie: MovieEntity)

    @Delete
    suspend fun deleteMovie(movie: MovieEntity)
}
```

Najdôležitejšie je pochopiť, že nad databázou nepracujete priamo z composable funkcie. Compose obrazovka pošle udalosť do `ViewModel` triedy, `ViewModel` zavolá repozitár a repozitár následne použije DAO.

4.3 Databáza

Trieda databázy spája entity a DAO do jedného celku. Označuje sa anotáciou `@Database`. V tomto cvičení budete mať iba jednu entitu a jedno DAO, preto je definícia databázy pomerne jednoduchá.

```
@Database(
    entities = [MovieEntity::class],
    version = 1,
    exportSchema = false
)
abstract class MovieDatabase : RoomDatabase() {
    abstract fun movieDao(): MovieDao
}
```

Keďže databázu nechcete vytvárať opakovane, spravidla sa používa singleton prístup. To znamená, že počas behu aplikácie pracujete s jednou zdieľanou inštanciou databázy.

5 Prepojenie Room s architektúrou aplikácie

Aplikácia v tomto cvičení bude mať jednoduchý tok údajov. Používateľ vykoná akciu na obrazovke. Compose obrazovka zavolá funkciu vo `ViewModel` triede. `ViewModel` spracuje logiku a deleguje databázovú operáciu na repozitár. Repozitár následne komunikuje s

DAO a Room databázou. Používateľské rozhranie rieši iba to, ako sa údaje zobrazujú. ViewModel rieši spracovanie udalostí a tvorbu stavu obrazovky. Dátová vrstva rieši trvalé uloženie údajov.

```
class MovieRepository(  
    private val movieDao: MovieDao  
) {  
    suspend fun getAllMovies(): List<MovieEntity> = movieDao.  
        getAllMovies()  
    suspend fun insertMovie(movie: MovieEntity) = movieDao.  
        insertMovie(movie)  
    suspend fun updateMovie(movie: MovieEntity) = movieDao.  
        updateMovie(movie)  
    suspend fun deleteMovie(movie: MovieEntity) = movieDao.  
        deleteMovie(movie)  
}
```

6 ViewModel ako most medzi UI a databázou

V tomto cvičení ostáva ViewModel hlavným stavovým držiteľom obrazovky. Rozdiel oproti CV8 je v tom, že už nepracuje len s výpočtom a validáciou, ale aj s databázovými operáciami. Po každej zmene v databáze znovu načíta aktuálny zoznam filmov a uloží ho do stavu obrazovky.

```
class MovieWatchlistViewModel(application: Application) :  
    AndroidViewModel(application) {  
  
    private val repository: MovieRepository  
  
    var uiState by mutableStateOf(MovieWatchlistUiState())  
        private set  
  
    init {  
        val movieDao = MovieDatabase.getDatabase(application).  
            movieDao()  
        repository = MovieRepository(movieDao)  
        loadMovies()  
    }  
}
```

Použitie AndroidViewModel v tomto cvičení zjednodušuje prístup k Application kontextu pri vytváraní databázy. V menších ukážkach je to praktické riešenie, pretože nemusíte zavádzať vlastnú Factory triedu. Dôležité je však pochopiť, že databáza nevzniká v composable funkcii, ale mimo používateľského rozhrania.

6.1 Pridanie filmu

Pri pridávaní filmu `ViewModel` najskôr skontroluje, či používateľ zadal neprázdny názov. Ak áno, vytvorí nový objekt `MovieEntity` a uloží ho do databázy.

```
fun addMovie() {
    val trimmedTitle = uiState.titleInput.trim()

    if (trimmedTitle.isBlank()) {
        uiState = uiState.copy(
            errorMessage = "Please enter a movie title."
        )
        return
    }

    viewModelScope.launch {
        repository.insertMovie(
            MovieEntity(
                title = trimmedTitle,
                createdAt = System.currentTimeMillis()
            )
        )

        uiState = uiState.copy(
            titleInput = "",
            errorMessage = null
        )

        loadMovies()
    }
}
```

Všimnite si dve dôležité veci. Po prvé, názov filmu sa validuje ešte pred zápisom. Po druhé, databázová operácia prebieha vo `viewModelScope.launch`, pretože ide o asynchrónnu prácu.

6.2 Označenie filmu ako videného

Keď používateľ označí film ako videný, otvorí sa dialóg, v ktorom zadá hodnotenie a komentár. Po potvrdení sa aktualizuje príslušný záznam v databáze. Zároveň sa vyplní `watchedAt`, aby bolo možné neskôr triediť filmy podľa času videnia.

```
fun saveReview() {
    val movieId = uiState.editingMovieId ?: return
    val comment = uiState.draftComment.trim()

    if (comment.isBlank()) {
        return
    }

    val movie = uiState.movies.firstOrNull { it.id == movieId }
        ?: return

    viewModelScope.launch {
        repository.updateMovie(
            movie.copy(
                isWatched = true,
                rating = uiState.draftRating,
                comment = comment,
                watchedAt = movie.watchedAt ?: System.
                    currentTimeMillis()
            )
        )

        uiState = uiState.copy(
            showReviewDialog = false,
            editingMovieId = null,
            draftRating = 3,
            draftComment = ""
        )

        loadMovies()
    }
}
```

Použitie výrazu `movie.watchedAt ?: System.currentTimeMillis()` zabezpečí, že pri prvom označení filmu za videný sa uloží čas videnia. Ak používateľ neskôr len upraví komentár alebo hodnotenie, pôvodný čas ostane zachovaný.

7 UiState pre celú obrazovku

Rovnako ako v CV8 je aj tu vhodné mať jednu stavovú dátovú triedu, ktorá reprezentuje aktuálny stav obrazovky. Vďaka tomu je stav centralizovaný a ľahko čitateľný. Okrem vstupného textu a zoznamu filmov bude obsahovať aj informácie o zvolenom filtri, zvolenom triedení a stave dialógu pre komentár.

```
data class MovieWatchlistUiState(
    val titleInput: String = "",
    val movies: List<MovieEntity> = emptyList(),
    val selectedFilter: MovieFilter = MovieFilter.ALL,
    val selectedSort: MovieSort = MovieSort.ADDED_NEWEST,
    val showReviewDialog: Boolean = false,
    val editingMovieId: Long? = null,
    val draftRating: Int = 3,
    val draftComment: String = "",
    val errorMessage: String? = null
)
```

8 Filtrovanie a triedenie

Filter a triedenie v tomto cvičení nie sú uložené v databáze ako samostatné tabuľky. Ide len o prezentačnú logiku aplikácie. Preto je vhodné držať ich v stave obrazovky a aplikovať ich nad zoznamom načítaným z databázy. V tomto príklade ale zároveň vidíte, že nie všetko musí byť nutne databázová operácia. Niektoré rozhodnutia patria do logiky obrazovky.

```
private fun filterAndSortMovies(
    movies: List<MovieEntity>,
    filter: MovieFilter,
    sort: MovieSort
): List<MovieEntity> {
    val filteredMovies = movies.filter { movie ->
        when (filter) {
            MovieFilter.ALL -> true
            MovieFilter.WATCHED -> movie.isWatched
            MovieFilter.UNWATCHED -> !movie.isWatched
        }
    }

    return when (sort) {
        MovieSort.ADDED_NEWEST -> filteredMovies.
            sortedByDescending { it.createdAt }
        MovieSort.WATCHED_NEWEST -> filteredMovies.sortedWith(
            compareByDescending<MovieEntity> { it.isWatched }
                .thenByDescending { it.watchedAt ?: 0L }
                .thenByDescending { it.createdAt }
        )
    }
}
```

Predvolené triedenie je podľa času pridania. Druhé triedenie používa čas označenia za videný, pričom videné filmy sa zobrazia vyššie.

9 Zobrazenie komentára iba v jednom riadku

Jednou z menších, ale praktických úloh v tomto cvičení je zobrazenie komentára len ako krátkeho náhľadu v zozname filmov. Používateľ tak dostane rýchlu informáciu bez toho, aby bola položka príliš vysoká.

V Compose sa to dá riešiť veľmi jednoducho cez `maxLines = 1` a `TextOverflow.Ellipsis`. Ak je komentár dlhší, zobrazí sa len jeho začiatok a zvyšok sa nahradí tromi bodkami.

```
Text(  
    text = movie.comment,  
    style = MaterialTheme.typography.bodyMedium,  
    maxLines = 1,  
    overflow = TextOverflow.Ellipsis  
)
```

10 Compose obrazovka a LazyColumn

Samotná obrazovka zostáva napísaná v Jetpack Compose. Horná časť obsahuje hlavičku, formulár na pridanie filmu a sekciu s filrami a triedením. Spodná časť využíva `LazyColumn`, pretože počet položiek sa môže meniť a zoznam má byť efektívne vykresľovaný.

```
LazyColumn(  
    modifier = Modifier.fillMaxSize(),  
    contentPadding = PaddingValues(bottom = 16.dp),  
    verticalArrangement = Arrangement.spacedBy(10.dp)  
) {  
    items(  
        items = visibleMovies,  
        key = { it.id }  
    ) { movie ->  
        MovieCard(  
            movie = movie,  
            onOpenReviewDialog = { onOpenReviewDialog(movie.id) },  
            onMarkAsUnwatchedClick = { onMarkAsUnwatchedClick(movie) },  
            onDeleteMovieClick = { onDeleteMovieClick(movie) }  
        )  
    }  
}
```

Použitie `key = { it.id }` je vhodné preto, že každá položka databázy má vlastný identifikátor. Compose tak vie lepšie rozlišovať jednotlivé položky zoznamu.

11 Pridanie Room a KSP konfigurácie

Aby bolo možné knižnicu Room v Kotlin projekte správne použiť, nestačí pridať iba runtime závislosti. Je potrebné doplniť aj KSP plugin, pretože práve ten zabezpečí vygenerovanie implementácie databázy počas buildu projektu.

Ak je projekt vytvorený s použitím version catalogs, je vhodné najprv doplniť verziu a plugin alias do súboru `libs.versions.toml`.

```
[versions]
ksp = "2.0.21-1.0.26"

[plugins]
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

Následne je potrebné deklarovať plugin aj v koreňovom súbore `build.gradle.kts`.

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.ksp) apply false
}
```

V modulovom súbore `app/build.gradle.kts` potom zapnete plugin KSP a doplníte potrebné závislosti pre Room a Compose ViewModel.

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
}

dependencies {
    implementation("androidx.room:room-runtime:2.8.4")
    implementation("androidx.room:room-ktx:2.8.4")
    ksp("androidx.room:room-compiler:2.8.4")

    implementation("androidx.lifecycle:lifecycle-viewmodel-
compose:2.8.6")
    implementation("androidx.lifecycle:lifecycle-runtime-compose
:2.8.6")
}
```

Po tejto konfigurácii vie Room počas buildu vytvoriť databázovú implementáciu, ktorú aplikácia následne používa pri spustení.

12 Zhrnutie

V tomto cvičení sa učíte prvýkrát pracovať s trvalým uložením údajov v Android aplikácii cez Room. Zároveň nadväzujete na predchádzajúce cvičenia, pretože Compose stále slúži na vykreslenie rozhrania, `ViewModel` stále drží stav obrazovky a udalosti od používateľa stále tečú rovnakým smerom. Novinkou je, že údaje už nezostávajú len v pamäti, ale ukladajú sa do databázy a vedia prežiť zatvorenie aplikácie.

V praxi si na tomto cvičení precvičíte definíciu entity, tvorbu DAO, vytvorenie databázy, jednoduchý repozitár, prácu s `ViewModel` triedou pri databázových operáciách a zobrazenie databázových dát v `LazyColumn`. Okrem toho si precvičíte aj malé používateľské detaily, napríklad filter, triedenie podľa času a jednoriadkový náhľad komentára.